COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	SUBALISID, JAPPETH ROYCE B. BAUTISTA, MARK JONEIL S.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 12.1	**Algorithmic Strategies**

1. PROCEDURE OUTPUT

Pre-Lab Tip:
When you are given a problem to solve using dynamic programming, the trick is “Not to over-think, because the solution is always hidden in the problem”. Most beginners make this mistake of thinking too much, only because they thought the problem was difficult and only a complex solution would solve it. But in most cases the solution turns out to be simple.

Before you start coding, formulate the problem (not the solution) mathematically on a piece of paper. Because a problem well formulated is a problem half solved.

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

For the pre-lab review, you must fill out the table below. Using algorithms implemented in the previous activities, identify which algorithm satisfies the given algorithmic strategy. For the analysis, you must explicitly provide what part of the algorithm you have identified matches the corresponding strategy.

Strategy	Algorithm	Analysis
Recursion	Recursive Binary Search	In finding an element in a sorted array, we used a function that takes the array, low, high, and data to find. This is so that we can call the function inside the function without losing the array and data, then we modify the low and high based on the conditions; (data < middle) or (data > middle)
Brute Force	Linear Search	Since we use linear search on any type of linked list or array, where we’re looking at every single element in the list, it is considered a brute force method.

Backtracking	Pre-order Traversal	Since we start on a tree/graph node then go from the bottom-left most child before moving back to the other branches to search for/print elements, this algorithm can be described as a backtracking algorithm.
Greedy	Selection Sort	Selection Sort acts as a greedy algorithm by making the locally optimal choice at each step, scanning the unsorted portion of the array to find and swap the single smallest remaining element to its final position.
Divide-and-Conquer	Merge Sort	Merge sort breaks arrays into smaller sub arrays using recursion until each sub array only has one element, sorting them during the process before merging them back together.

Table 12-1. Algorithmic Strategies and Examples

The Problem: MINIMUM STEPS TO ONE

On a positive integer, you can perform any one of the following 3 steps.

1.) Subtract 1 from it. (n = n - 1)
2.) If its divisible by 2, divide by 2. (If n % 2 == 0, then n = n / 2)
3.) If its divisible by 3, divide by 3. (If n % 3 == 0, then n = n / 3).
Now the question is, given a positive integer n, find the minimum number of steps that takes n to 1.

Examples:
1.) For n = 1, output: 0
2.) For n = 4, output: 2 (4 /2 = 2 /2 = 1)
3.) For n = 7, output: 3 (7 -1 = 6 /3 = 2 /2 = 1)

One can think of greedily choosing the step, which makes n as low as possible and continue the same, till it reaches 1. If you observe carefully, the greedy strategy doesn't work here.

Eg: Given n = 10 , Greedy --> 10 /2 = 5 -1 = 4 /2 = 2 /2 = 1 (4 steps).

But the optimal way is --> 10 -1 = 9 /3 = 3 /3 = 1 (3 steps).

So, we need to try out all possible steps we can make for each possible value of n we encounter and choose

the minimum of these possibilities.

It all starts with recursion.

$$F(n) = 1 + min\{ F(n-1) , F(n/2) , F(n/3) \} \; if \; (n > 1) \; , else \; 0 \; (\; i.e. \; , F(1) = 0 \;) \; .$$

Now that we have our recurrence equation, we can right away start coding the recursion. Wait. Does it have over-lapping subproblems? YES. Is the optimal solution to a given input depends on the optimal solution of its subproblems? Yes... Bingo! It is dynamic programming. So, we just store the solutions to the subproblems we solve and use them later, as in memorization, or we start from the bottom and move up till the given n, as in dynamic programming. As it's the very first problem we are looking at here, let's see both the codes:

Memoization

```
int memo[100];
int getMinSteps(int n) {
        if (n == 1) {
            return 0;
        }
        if (memo[n] != -1) {
            return memo[n];
        }
        int r = 1 + getMinSteps(n - 1);
        if(n%2 == 0) {
            r = std::min(r, 1 + getMinSteps(n/2));
        }
        if(n%3 == 0) {
            r = std::min(r, 1 + getMinSteps(n/2));
        }
        memo[n] = r;
        return r;
}

int main() {
    for(int i = 0; i < 100; i++) {
        memo[i] = -1;
    }

    std::cout << "\nSteps to 1: " << getMinSteps(1);
    std::cout << "\nSteps to 4: " << getMinSteps(4);
    std::cout << "\nSteps to 7: " << getMinSteps(7);

    return 0;
}
```

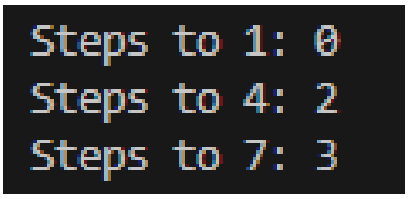
Screenshot	
Analysis	The first call (getMinSteps(1)) calls the base case and returns immediately. The second call uses 4 as the argument, first it gets the get min steps with 3 since we use r = 1 + getMinSteps(n-1) and 4-1 = 3, so it satisfies the n%3 first. Then it gets a result of 1 step. Then it gets the result for 2 since we call the function again with n-1 = 2 now, then it also gets a 1 as the result. Comparing the results with the min function, we get an answer of 2 on both instances so it outputs 2. Finally, with 7, we get the n%3 first since its n-1=6 which results in the lowest steps(2) outputting 1 + getminsteps(6) = 3.

Table 12-2. Memoization Implementation

Bottom-Up DP

```
int getMinSteps ( int n )
{
int dp[n+1], i;
dp[1] = 0; // trivial case
for( i = 2 ; i < = n ; i ++ )
{
dp[i] = 1 + dp[i-1];
if(i%2==0) dp[i] = min( dp[i] , 1+ dp[i/2] );
if(i%3==0) dp[i] = min( dp[i] , 1+ dp[i/3] );
}
return dp[n];
}
```

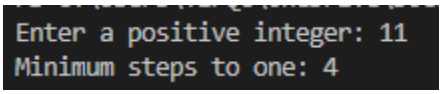
Screenshot	
Analysis	The Bottom Up DP algorithm computes the minimum steps to reduce positive integer n to 1 by building the optimal solution iteratively from base case 1 up to n. Beginning with the base case dp[1] = 0, the algorithm fills a lookup table where each entry dp[i] evaluates the minimum steps among subtracting 1, dividing by 2 and dividing by 3 if divisible. By systematically storing and reusing subproblem solutions.

Table 12-3. Bottom-Up Dynamic Programming Implementation

Both the approaches are fine. But one should also take care of the lot of overhead involved in the function calls in Memoization, which may give StackOverFlow error or TLE rarely.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

3. CONCLUSION

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.